

## 12.5-CSP基础专题总复盘（01-12）

## 12.5-CSP基础专题总复盘（01-12）

### 使用说明

---

这篇不是完整讲义，而是考前速查笔记。

重点看三件事：

1. 这个函数 / 操作是什么意思
  2. 一般怎么用
  3. 容易在哪里写错
- 

## 01-STL基础

### vector

---

```
vector<int> a;
```

定义一个空的动态数组。

适合：一开始不知道有多少个元素，后面用 `push_back` 慢慢加入。

注意：此时 `a` 里面没有元素，不能直接写 `a[0]=x`。

---

```
vector<int> a(n);
```

定义一个长度为 `n` 的数组。

每个元素默认是 `0`。

适合：一开始就知道数组长度，并且后面要用下标访问。

---

```
vector<int> a(n,x);
```

定义一个长度为 `n` 的数组。

每个元素初始值都是 `x`。

例如：

```
vector<int> a(5,10);
```

表示有 5 个元素，每个都是 10。

---

```
a.push_back(x);
```

把 `x` 加到 vector 的最后面。

这是“新增元素”，不是修改已有元素。

---

```
a[i]
```

访问第 `i` 个元素。

注意：必须保证第 `i` 个位置已经存在，否则会越界。

---

```
a.size()
```

返回 vector 当前元素个数。

常用于遍历：

```
for(int i=0;i<a.size();i++)
{
    cout<<a[i]<<" ";
}
```

注意：`a.size()` 返回的不是 `int`，但 CSP 里通常这样写没问题。

---

```
a.empty()
```

判断 vector 是否为空。

常用于访问前保护：

```
if(!a.empty())
{
    cout<<a.back();
}
```

---

```
a.clear()
```

清空 vector 里的所有元素。

清空后：

```
a.size()
```

变成 0。

---

```
a.resize(n);
```

把 vector 的长度改成 n。

如果原来长度不够，会补默认值 0。

如果原来长度超过 n，后面的元素会被删掉。

---

```
a.back()
```

访问最后一个元素。

注意：使用前要保证 vector 不是空的。

---

```
a.pop_back();
```

删除最后一个元素。

注意：pop\_back() 不返回元素，只负责删除。

错误写法：

```
int x=a.pop_back();
```

正确写法：

```
int x=a.back();  
a.pop_back();
```

---

## 二维 vector

---

```
vector<vector<int>> a;
```

定义一个空的二维 vector。

此时没有任何行，不能直接访问 `a[0]`。

---

```
vector<vector<int>> a(n);
```

定义一个有 `n` 行的二维 vector。

但是每一行都是空的。

可以这样加元素：

```
a[i].push_back(x);
```

---

```
vector<vector<int>> a(n,vector<int>(m));
```

定义一个 `n` 行 `m` 列的二维 vector。

每个元素默认是 `0`。

可以直接访问：

```
a[i][j]
```

---

```
vector<vector<int>> a(n,vector<int>(m,x));
```

定义一个 `n` 行 `m` 列的二维 vector。

每个元素初始值都是 `x`。

---

```
a[i].push_back(x);
```

给第 `i` 行的末尾添加一个元素。

前提：第 `i` 行已经存在。

---

```
a.push_back(row);
```

给二维 vector 添加一整行。

其中 `row` 是一个 `vector<int>`。

例如：

```
vector<vector<int>> a;  
vector<int> row;  
  
row.push_back(1);  
row.push_back(2);  
row.push_back(3);  
  
a.push_back(row);
```

---

## map

---

```
map<string,int> mp;
```

定义一个从 `string` 到 `int` 的映射。

常用于统计字符串出现次数。

---

```
map<int, int> mp;
```

定义一个从 `int` 到 `int` 的映射。

常用于统计数字出现次数。

---

```
mp[x]
```

访问 key 为 `x` 的值。

注意：如果 `x` 不存在，`mp[x]` 会自动创建，并且初始值为 `0`。

---

```
mp[x]++;
```

统计 `x` 出现次数。

如果 `x` 第一次出现，会先创建 `mp[x]=0`，然后再加一。

---

```
mp.count(x)
```

判断 key `x` 是否存在。

存在返回 `1`，不存在返回 `0`。

如果只是判断是否出现过，优先用这个，不要乱用 `mp[x]`。

---

```
mp.erase(x);
```

删除 key 为 `x` 的记录。

---

```
mp.size()
```

返回 map 中有多少个不同的 key。

---

```
mp.empty()
```

判断 map 是否为空。

---

```
for(auto x:mp)
{
    cout<<x.first<<" "<<x.second<<"\n";
}
```

遍历 map。

`x.first` 是 key。

`x.second` 是 value。

注意：map 会按照 key 自动从小到大排序，不是按照插入顺序。

---

## set

---

```
set<int> s;
```

定义一个 int 集合。

特点：

自动去重  
自动排序

---

```
s.insert(x);
```

插入元素 `x`。

如果 `x` 已经存在，不会重复插入。

---

```
s.count(x)
```

判断 `x` 是否存在。

存在返回 `1`，不存在返回 `0`。

---

```
s.erase(x);
```

删除元素 `x`。

---

```
s.size()
```

返回 set 中元素个数。

因为 set 会去重，所以这个值是不同元素的数量。

---

```
s.empty()
```

判断 set 是否为空。

---

```
for(auto x:s)
{
    cout<<x<<" ";
}
```

遍历 set。

注意：输出顺序是从小到大，不是插入顺序。

---

## pair

---

```
pair<int,int> p;
```

定义一个存两个 int 的二元组。

---



```
p.first
```

访问第一个值。

---

```
p.second
```

访问第二个值。

---

```
pair<int,int> p={1,2};
```

定义并初始化。

此时：

```
p.first = 1  
p.second = 2
```

---

```
make_pair(1,2)
```

创建一个 pair。

例如：

```
p=make_pair(1,2);
```

---

## struct

---

```
struct student  
{  
    string name;  
    int score;  
};
```

定义一个结构体类型。

适合存多个相关属性。

---

```
student s;
```

定义一个结构体变量。

---

```
s.name
```

访问结构体里的 `name` 成员。

---

```
s.score
```

访问结构体里的 `score` 成员。

---

```
student a[100];
```

定义结构体数组。

常用于排序题、模拟题、贪心题。

---

## auto

---

```
auto x=5;
```

让编译器自动判断变量类型。

---

```
for(auto x:a)
{
    cout<<x<<" ";
}
```

遍历 vector / set 等容器。

```
for(auto x:mp)
{
    cout<<x.first<<" "<<x.second<<"\n";
}
```

遍历 map。

`x.first` 是 key。

`x.second` 是 value。

## 02-STL提高

### queue

```
queue<int> q;
```

定义一个队列。

特点：

先进先出

常用于 BFS、排队模拟、层序遍历。

```
q.push(x);
```

把 `x` 放到队尾。

```
q.front()
```

访问队首元素。

注意：使用前要保证队列不为空。

---

```
q.pop();
```

删除队首元素。

注意： `pop()` 不返回元素。

正确取出队首：

```
int x=q.front();  
q.pop();
```

---

```
q.empty()
```

判断队列是否为空。

常用于循环：

```
while(!q.empty())  
{  
    int x=q.front();  
    q.pop();  
}
```

---

```
q.size()
```

返回队列中元素个数。

---

## stack

---

```
stack<int> st;
```

定义一个栈。

特点：

```
后进先出
```

常用于括号匹配、表达式处理、DFS 非递归、单调栈。

---

```
st.push(x);
```

把 `x` 压入栈顶。

---

```
st.top()
```

访问栈顶元素。

注意：使用前要保证栈不为空。

---

```
st.pop();
```

删除栈顶元素。

注意：`pop()` 不返回元素。

正确取出栈顶：

```
int x=st.top();  
st.pop();
```

```
st.empty()
```

判断栈是否为空。

---

```
st.size()
```

返回栈中元素个数。

---

## deque

---

```
deque<int> dq;
```

定义一个双端队列。

特点：

两头都可以插入和删除

---

```
dq.push_back(x);
```

从后面加入元素。

---

```
dq.push_front(x);
```

从前面加入元素。

---

```
dq.pop_back();
```

删除最后一个元素。

---

```
dq.pop_front();
```

删除第一个元素。

---

```
dq.front()
```

访问第一个元素。

---

```
dq.back()
```

访问最后一个元素。

---

```
dq.empty()
```

判断 deque 是否为空。

---

```
dq.size()
```

返回元素个数。

---

## priority\_queue

---

```
priority_queue<int> q;
```

定义一个大根堆。

默认 `top()` 是当前最大值。

---

```
q.push(x);
```

加入元素 `x`。

---

```
q.top()
```

访问堆顶元素。

大根堆中，堆顶是最大值。

小根堆中，堆顶是最小值。

---

```
q.pop();
```

删除堆顶元素。

注意： `pop()` 不返回元素。

正确写法：

```
int x=q.top();  
q.pop();
```

```
q.empty()
```

判断优先队列是否为空。

```
q.size()
```

返回元素个数。

```
priority_queue<int,vector<int>,greater<int>>> q;
```

定义一个小根堆。

`top()` 是当前最小值。

常用于合并果子、Dijkstra 等。

---

## lower\_bound

```
lower_bound(a.begin(),a.end(),x)
```

在有序 vector 中，找到第一个 `>= x` 的位置。

返回的是迭代器，不是下标。

---



```
int pos=lower_bound(a.begin(),a.end(),x)-a.begin();
```

把 `lower_bound` 的结果转成下标。

---

```
lower_bound(a,a+n,x)
```

在普通数组中，找到第一个 `>= x` 的位置。

---

```
int pos=lower_bound(a,a+n,x)-a;
```

普通数组中求下标。

---

## upper\_bound

---

```
upper_bound(a.begin(),a.end(),x)
```

在有序 vector 中，找到第一个 `> x` 的位置。

---

```
int pos=upper_bound(a.begin(),a.end(),x)-a.begin();
```

求下标。

---

```
upper_bound(a,a+n,x)
```

在普通数组中，找到第一个 `> x` 的位置。

---

## lower\_bound / upper\_bound 组合

---

```
int cnt=upper_bound(a.begin(),a.end(),x)-lower_bound(a.begin(),a.end(),x);
```

统计 `x` 出现了几次。

---

```
int cnt=upper_bound(a.begin(),a.end(),R)-lower_bound(a.begin(),a.end(),L);
```

统计 `[L,R]` 中有多少个数。

---

## algorithm 常用函数

---

```
sort(a.begin(),a.end());
```

对 vector 从小到大排序。

---

```
sort(a,a+n);
```

对普通数组从小到大排序。

---

```
reverse(a.begin(),a.end());
```

反转 vector。

---

```
reverse(s.begin(),s.end());
```

反转字符串。

---

```
max(a,b)
```

返回较大值。

---

```
min(a,b)
```

返回较小值。

---

```
swap(a,b)
```

交换两个变量的值。

---

```
count(a.begin(),a.end(),x)
```

统计 `x` 在容器中出现了多少次。

注意：这是线性扫描，复杂度是 `O(n)`。

---

```
find(a.begin(),a.end(),x)
```

查找 `x` 是否存在。

如果找到，返回对应迭代器。

如果找不到，返回 `a.end()`。

---

```
fill(a,a+n,0);
```

把普通数组某一段全部赋值为 0。

---

```
fill(a.begin(),a.end(),0);
```

把 vector 全部赋值为 0。

---

```
unique(a.begin(),a.end())
```

把相邻重复元素移到后面，返回去重后的新末尾。

注意： `unique` 不会真正删除元素。

标准写法：

```
sort(a.begin(), a.end());  
a.erase(unique(a.begin(), a.end()), a.end());
```

---

## 03-排序

### 普通数组排序

---

```
sort(a, a+n);
```

把普通数组 `a[0]` 到 `a[n-1]` 从小到大排序。

注意：排序范围是 `[a, a+n)`，右边界不包含。

---

### vector 排序

---

```
sort(a.begin(), a.end());
```

把整个 vector 从小到大排序。

---

### 降序排序

---

```
sort(a.begin(), a.end(), greater<int>());
```

把 vector 从大到小排序。

需要头文件：

```
#include<functional>
```

如果使用万能头，则不用单独加。

---

## 自定义排序 cmp

---

```
bool cmp(int a,int b)
{
    return a<b;
}
```

升序排序。

含义：

如果  $a < b$ ，就让  $a$  排在  $b$  前面。

---

```
bool cmp(int a,int b)
{
    return a>b;
}
```

降序排序。

含义：

如果  $a > b$ ，就让  $a$  排在  $b$  前面。

---

```
sort(a,a+n,cmp);
```

普通数组使用自定义排序。

---

```
sort(a.begin(),a.end(),cmp);
```

vector 使用自定义排序。

---

## cmp 的核心理解

---

```
bool cmp(A a,A b)
{
    return 条件;
}
```

如果 条件 为真，表示：

a 应该排在 b 前面。

不要理解成“要不要交换”。

---

## pair 排序

---

```
vector<pair<int,int>> a;
sort(a.begin(),a.end());
```

pair 默认排序规则：

先按 first 从小到大排  
如果 first 相同，再按 second 从小到大排

---

## struct 排序

---

```
struct student
{
    string name;
    int score;
};
```

定义学生结构体。

---

```
bool cmp(student a,student b)
{
    return a.score>b.score;
}
```

按照分数从高到低排序。

```
sort(a, a+n, cmp);
```

对结构体数组排序。

## 多关键字排序

```
bool cmp(student a, student b)
{
    if(a.score != b.score)
    {
        return a.score > b.score;
    }
    else
    {
        return a.id < b.id;
    }
}
```

含义：

先按分数从高到低排；  
如果分数相同，按编号从小到大排。

## cmp 易错点

```
return a <= b;
```

不推荐。

排序比较函数里不要写 `<=` 或 `>=`。

应该写：

```
return a < b;
```

或者：

```
return a > b;
```

---

## 04-高精度

### 字符转数字

---

```
int x=s[i]-'0';
```

把字符数字转成真正的整数。

例如 '7' 变成 7。

---

### 数字转字符

---

```
char c=x+'0';
```

把数字转成字符。

例如 7 变成 '7'。

---

### 高精度加法核心循环

---

```
while(i>=0 || j>=0 || carry)
```

只要：

```
a 还没加完  
b 还没加完  
还有进位
```

就继续计算。

---

```
int sum=x+y+carry;
```

当前位相加，再加上进位。

---



```
ans.push_back(sum%10+'0');
```

当前位留下个位数字。

因为 `ans` 是 string，所以要加 `'0'` 转成字符。

---

```
carry=sum/10;
```

更新进位。

---

```
reverse(ans.begin(),ans.end());
```

因为计算时从低位到高位，答案是反的，所以最后要反转。

---

## 高精度减法比较大小

---

```
bool smaller(string a,string b)
{
    if(a.size()!=b.size())
    {
        return a.size()<b.size();
    }

    return a<b;
}
```

判断大整数字符串 `a` 是否小于 `b`。

长度不同，长度短的数更小。

长度相同，可以直接按字符串比较。

---

## 高精度减法借位

---

```
x-=borrow;
```

先减去上一位借走的数。

```
if(x<y)
{
    x+=10;
    borrow=1;
}
else
{
    borrow=0;
}
```

如果当前位不够减，就向高位借 1。

```
ans.push_back(x-y+'0');
```

保存当前位结果。

```
while(ans.size()>1 && ans.back()=='0')
{
    ans.pop_back();
}
```

去掉前导 0。

因为答案是反着存的，所以前导 0 在 `ans.back()`。

注意：要保留至少一个字符，不能把结果 0 删空。

## 高精度乘法

```
for(int i=a.size()-1;i>=0;i--)
{
    A.push_back(a[i]-'0');
}
```

把字符串数字倒序存入 vector。

低位存在前面，方便进位。

```
C[i+j] += A[i] * B[j];
```

反向存储时，`A[i]` 是  $10^i$  位，`B[j]` 是  $10^j$  位。

相乘后贡献到  $10^{(i+j)}$  位，所以加到 `C[i+j]`。

```
C[i+1] += C[i] / 10;  
C[i] %= 10;
```

统一处理进位。

```
while(C.size() > 1 && C.back() == 0)  
{  
    C.pop_back();  
}
```

去掉前导 0。

## 高精度乘低精度

```
int t = a[i] * b + carry;
```

当前位乘普通整数，再加进位。

```
c.push_back(t % 10);
```

当前位留下个位。

```
carry = t / 10;
```

更新进位。

```
while(carry)
{
    c.push_back(carry%10);
    carry/=10;
}
```

处理最后剩余进位。

## 05-前缀和

### 一维前缀和定义

```
s[i]=s[i-1]+a[i];
```

`sum[i]` 表示前 `i` 个数的和。

建议数组从 1 开始存，这样区间公式更自然。

### 一维区间求和

```
s[r]-s[l-1]
```

求 `[l,r]` 的区间和。

含义：

```
前 r 个数的和
减去
前 l-1 个数的和
```

剩下的就是 `[l,r]`。

### 一维前缀和初始化

```
int s[100]={0};
```

保证  $s[0]=0$ 。

如果  $s[0]$  乱了，后面区间查询会错。

---

## 二维前缀和定义

```
s[i][j]=s[i-1][j]+s[i][j-1]-s[i-1][j-1]+a[i][j];
```

$s[i][j]$  表示从  $(1,1)$  到  $(i,j)$  这个矩形区域的和。

---

## 为什么要减 $s[i-1][j-1]$

```
s[i-1][j]+s[i][j-1]
```

左上角区域被加了两次。

所以要减掉一次：

```
-s[i-1][j-1]
```

---

## 二维矩形求和

```
s[x2][y2]-s[x1-1][y2]-s[x2][y1-1]+s[x1-1][y1-1]
```

求  $(x1,y1)$  到  $(x2,y2)$  这个矩形的和。

理解：

大矩形

-

上面不要的

-

左边不要的

+

左上角被减重了，补回来

---

## 06-差分

### 一维差分定义

---

```
d[i]=a[i]-a[i-1];
```

差分数组记录的是相邻元素之间的变化量。

---

### 一维区间加法

---

```
d[l]+=c;  
d[r+1]-=c;
```

表示给 `[l, r]` 整体加 `c`。

含义：

```
从 l 开始多 c;  
从 r+1 开始不再多 c。
```

---

### 一维差分恢复

---

```
for(int i=1;i<=n;i++)  
{  
    d[i]+=d[i-1];  
}
```

对差分数组做前缀和，恢复出最终数组。

---

### 原数组建差分

---

```
d[1]=a[1];  
  
for(int i=2;i<=n;i++)  
{
```

```
d[i]=a[i]-a[i-1];  
}
```

如果初始数组不是全 0，需要先建差分。

---

## 二维差分矩形加法

```
d[x1][y1]+=c;  
d[x2+1][y1]-=c;  
d[x1][y2+1]-=c;  
d[x2+1][y2+1]+=c;
```

表示给矩形  $(x1, y1)$  到  $(x2, y2)$  全部加  $c$ 。

---

## 二维差分恢复

```
d[i][j]=d[i-1][j]+d[i][j-1]-d[i-1][j-1]+d[i][j];
```

本质就是二维前缀和。

恢复后， $d[i][j]$  就是最终矩阵的值。

---

## 差分易错点

```
d[r+1]-=c;
```

这里是  $r+1$ ，不是  $r$ 。

```
d[x2+1][y2+1]+=c;
```

右下角是加回来，不是减。

---

数组要多开一圈，因为会访问  $r+1$ 、 $x2+1$ 、 $y2+1$ 。

---

## 07-贪心

### 贪心核心

---

每一步都选择当前看起来最优的方案。

关键不是代码，而是证明：

当前这样选不会吃亏。

---

### 部分背包单位价值

---

```
a[i].unit=(double)a[i].value/a[i].weight;
```

计算单位价值。

---

```
bool cmp(item a,item b)
{
    return a.unit>b.unit;
}
```

按单位价值从高到低排序。

适合部分背包，因为物品可以拿一部分。

---

### 排队接水排序

---

```
bool cmp(person a,person b)
{
    return a.time<b.time;
}
```

时间短的人排前面。

这样可以让后面等待的人更少，总等待时间更小。

---



## 合并果子小根堆

```
priority_queue<int, vector<int>, greater<int>> q;
```

每次取出最小的两堆。

```
int a=q.top();
q.pop();

int b=q.top();
q.pop();

ans+=a+b;
q.push(a+b);
```

每次合并最小两堆，并把新堆放回去。

## 纪念品分组双指针

```
sort(a,a+n);
```

先排序。

```
int l=0, r=n-1;
```

`l` 指向最轻的人，`r` 指向最重的人。

```
if(a[l]+a[r]<=w)
{
    l++;
    r--;
}
else
{
    r--;
}
```

如果最轻和最重能一起走，就配对。

如果不能，最重的人只能自己一组。

---

## 08-二分基础与边界

### 普通二分查找

---

```
int l=0,r=n-1;

while(l<=r)
{
    int mid=(l+r)/2;

    if(a[mid]==x)
    {
        cout<<"找到";
        break;
    }
    else if(a[mid]<x)
    {
        l=mid+1;
    }
    else
    {
        r=mid-1;
    }
}
```

在有序数组中查找 `x`。

---

**`l=mid+1`**

```
l=mid+1;
```

表示 `a[mid]` 太小，答案只能在右边。

---

**`r=mid-1`**

```
r=mid-1;
```

表示 `a[mid]` 太大，答案只能在左边。

---

## 手写 lower\_bound

---

```
int lower(vector<int> &a, int x)
{
    int l=0, r=a.size();

    while(l<r)
    {
        int mid=(l+r)/2;

        if(a[mid]>=x)
        {
            r=mid;
        }
        else
        {
            l=mid+1;
        }
    }

    return l;
}
```

返回第一个 `>= x` 的位置。

注意：`r=a.size()`，因为答案可能是 `a.size()`，表示不存在这样的元素。

---

## 手写 upper\_bound

---

```
int upper(vector<int> &a, int x)
{
    int l=0, r=a.size();

    while(l<r)
    {
        int mid=(l+r)/2;

        if(a[mid]>x)
        {
            r=mid;
        }
        else
```

```

        {
            l=mid+1;
        }
    }

    return l;
}

```

返回第一个 `> x` 的位置。

## lower\_bound 判断存在

```

int pos=lower_bound(a.begin(),a.end(),x)-a.begin();

if(pos<a.size() && a[pos]==x)
{
    cout<<"存在";
}
else
{
    cout<<"不存在";
}

```

注意必须先判断：

```
pos<a.size()
```

否则可能越界。

## 统计某个数出现次数

```
int cnt=upper_bound(a.begin(),a.end(),x)-lower_bound(a.begin(),a.end(),x);
```

适合有序数组。

## 09-答案二分

### 答案二分核心模板：最大化答案

```
int l=0,r=1000000000;
int ans=0;

while(l<=r)
{
    int mid=(l+r)/2;

    if(check(mid))
    {
        ans=mid;
        l=mid+1;
    }
    else
    {
        r=mid-1;
    }
}
```

适合：

答案要尽量大  
check(mid) 为 true 表示 mid 可行

例如：

最大化最小距离  
锯子高度尽量高

---

## 答案二分核心模板：最小化答案

---

```
int l=0,r=1000000000;
int ans=0;

while(l<=r)
{
    int mid=(l+r)/2;

    if(check(mid))
    {
        ans=mid;
        r=mid-1;
    }
    else
    {

```

```
        l=mid+1;
    }
}
```

适合：

答案要尽量小  
check(mid) 为 true 表示 mid 可行

例如：

最小化最大段和  
最小化最大间距

---

## check(mid)

```
bool check(int mid)
{
    // 判断 mid 是否可行
}
```

check(mid) 的核心问题是：

如果答案是 mid，能不能做到？

写 check 前先问：

mid 表示什么？  
什么叫可行？

---

## P2678 跳石头 check

mid 表示最小跳跃距离至少为 mid。

如果某块石头离上一块保留石头太近，就移走它。

最后判断移走数量是否不超过限制。

---

## P1182 数列分段 check

`mid` 表示每一段允许的最大段和。

从左到右贪心分段。

当前段放不下就新开一段。

最后判断段数是否不超过 `m`。

## P3853 路标设置新增数量

$(d-1)/mid$

表示距离为 `d` 的两路标之间，为了让每段不超过 `mid`，需要新增多少个路标。

## 答案二分易错点

先确定：

要求最大，还是要求最小？

如果要求最大，通常：

```
if(check(mid))
{
    ans=mid;
    l=mid+1;
}
```

如果要求最小，通常：

```
if(check(mid))
{
    ans=mid;
    r=mid-1;
}
```

## 10-模拟

### 四方向数组

---

```
int dx[4]={-1,1,0,0};
int dy[4]={0,0,-1,1};
```

表示上下左右四个方向。

---

```
for(int k=0;k<4;k++)
{
    int nx=x+dx[k];
    int ny=y+dy[k];

    if(nx>=0 && nx<n && ny>=0 && ny<m)
    {
        // 合法位置
    }
}
```

遍历当前位置的四个相邻位置。

注意：一定要先判断合法，再访问数组。

---

### 八方向数组

---

```
int dx[8]={-1,-1,-1,0,0,1,1,1};
int dy[8]={-1,0,1,-1,1,-1,0,1};
```

常用于扫雷、八邻域统计。

---

### 标记数组

---

```
int mark[105][105]={0};
```

记录哪些位置需要修改。

适合：



先判断，后统一修改

例如消消乐。

```
if(mark[i][j])
{
    a[i][j]=0;
}
```

最后统一处理被标记的位置。

## 连续段扫描

```
int start=0;

while(start<m)
{
    int end=start;

    while(end<m && a[i][end]==a[i][start])
    {
        end++;
    }

    if(end-start>=3)
    {
        for(int k=start;k<end;k++)
        {
            mark[i][k]=1;
        }
    }

    start=end;
}
```

用于统计一行中连续相同的段。

比手动维护 `last/cnt` 更稳。

## 闰年判断

```
bool leap(int y)
{
    if(y%400==0)
    {
        return true;
    }

    if(y%4==0 && y%100!=0)
    {
        return true;
    }

    return false;
}
```

判断某年是否是闰年。

---

## 月份天数数组

---

```
int month[13]={0,31,28,31,30,31,30,31,31,30,31,30,31};
```

月份从 1 开始。

如果是闰年，要把 2 月改成 29。

---

## 日期推进一天

---

```
d++;

if(d>month[m])
{
    d=1;
    m++;
}

if(m>12)
{
    m=1;
    y++;
}
```

把日期往后推一天。

---

## 取模处理环形

---

```
pos=(pos+k)%n;
```

向后走 `k` 步。

```
pos=(pos-k+n)%n;
```

向前走 `k` 步。

加 `n` 是为了防止出现负数。

---

## 11-字符串

### string 定义

---

```
string s;
```

定义字符串。

```
cin>>s;
```

读入不含空格的字符串。

```
getline(cin,s);
```

读入一整行字符串，可以包含空格。

如果前面用了 `cin>>`，可能需要：

```
cin.ignore();
```

---

## 字符访问

---

```
s[i]
```

访问第 `i` 个字符。

注意：合法下标是 `0` 到 `s.size()-1`。

---

## 字符串长度

---

```
s.size()
```

返回字符串长度。

---

```
s.length()
```

也返回字符串长度。

通常和 `size()` 一样用。

---

## 字符串拼接

---

```
s+=t;
```

把字符串 `t` 接到 `s` 后面。

---

```
s.push_back(c);
```

把字符 `c` 加到字符串末尾。

注意：`push_back` 只能加字符，不能加字符串。

错误：

```
s.push_back("a");
```

正确：

```
s.push_back('a');
```

---

## 字符判断

---

```
if(c>='0' && c<='9')
```

判断是否是数字字符。

```
if(c>='a' && c<='z')
```

判断是否是小写字母。

```
if(c>='A' && c<='Z')
```

判断是否是大写字母。

---

## 字符和数字转换

---

```
int x=c-'0';
```

字符数字转整数。

```
char c=x+'0';
```

整数转字符数字。

---

## substr

---

```
s.substr(pos, len)
```

从下标 `pos` 开始，截取长度为 `len` 的子串。

注意：第二个参数是长度，不是结束下标。

---

```
s.substr(pos)
```

从 `pos` 开始一直截取到字符串末尾。

---

## find

---

```
s.find(t)
```

在 `s` 中查找子串 `t` 第一次出现的位置。

找到返回位置。

找不到返回：

```
string::npos
```

判断写法：

```
if(s.find(t)!=string::npos)
{
    cout<<"找到了";
}
```

## erase

---

```
s.erase(pos, len);
```

从 `pos` 开始删除 `len` 个字符。

---

## insert

---

```
s.insert(pos,t);
```

在 `pos` 位置前插入字符串 `t`。

---

## replace

---

```
s.replace(pos,len,t);
```

从 `pos` 开始，把 `len` 个字符替换成字符串 `t`。

---

## reverse 字符串反转

---

```
reverse(s.begin(),s.end());
```

把字符串反转。

---

## sort 字符串排序

---

```
sort(s.begin(),s.end());
```

把字符串中的字符排序。

---

## 回文判断

---

```
bool check(string s)
{
    int l=0;
    int r=s.size()-1;

    while(l<r)
    {
        if(s[l]!=s[r])
        {
            return false;
        }
    }
}
```

```
    }  
  
    l++;  
    r--;  
}  
  
return true;  
}
```

判断字符串是否是回文。

---

## 12-位运算

### 按位与

---

```
x&y
```

两个二进制位都为 1，结果才为 1。

```
if(x&1)  
{  
    cout<<"奇数";  
}  
else  
{  
    cout<<"偶数";  
}
```

判断奇偶。

最后一位是 1，就是奇数。

最后一位是 0，就是偶数。

---

### 按位或

---

```
x|y
```

两个二进制位只要有一个是 1，结果就是 1。



```
x=x|(1<<k);
```

把第  $k$  位变成 1。

## 按位异或

```
x^y
```

两个二进制位不同为 1，相同为 0。

```
x^x
```

结果是 0。

```
x^0
```

结果是  $x$ 。

```
ans=ans^a[i];
```

常用于抵消成对出现的数。

## 按位取反

```
~x
```

把二进制每一位取反。

注意：C++ 中整数用补码存储，初学阶段不要乱用。

## 左移

---

```
x<<k
```

把 `x` 的二进制左移 `k` 位。

对于非负整数，通常相当于：

```
x * 2^k
```

```
1<<k
```

表示  $2^k$ 。

注意：如果 `k` 很大，写：

```
1LL<<k
```

防止 int 溢出。

---

## 右移

---

```
x>>k
```

把 `x` 的二进制右移 `k` 位。

对于非负整数，通常相当于：

```
x / 2^k
```

向下取整。

---

## 判断第 `k` 位是不是 1

---

```
if(x & (1<<k))  
{
```

```
cout<<"第 k 位是 1";  
}
```

注意：第 0 位是最低位，也就是最右边那一位。

---

## 把第 k 位变成 1

---

```
x=x|(1<<k);
```

## 把第 k 位变成 0

---

```
x=x&(~(1<<k));
```

## 反转第 k 位

---

```
x=x^(1<<k);
```

如果第 k 位原来是 1，就变成 0。

如果原来是 0，就变成 1。

---

## 枚举所有子集状态

---

```
for(int s=0;s<(1<<n);s++)  
{  
    // s 表示一种选择状态  
}
```

如果有  $n$  个物品，每个物品选或不选，一共有  $2^n$  种状态。

---

## 判断状态中第 i 个物品是否被选

---

```
if(s&(1<<i))
{
    // 第 i 个物品被选了
}
```

---

## 高频易错点汇总

### vector 越界

---

```
vector<int> a;
a[0]=1;
```

错误。

因为 `a` 还没有元素。

---

### map 自动创建

---

```
mp[x]
```

如果 `x` 不存在，会自动创建。

判断存在时优先用：

```
mp.count(x)
```

---

### queue / stack / priority\_queue 的 pop

---

```
q.pop();
```

只删除，不返回元素。

正确：

```
int x=q.front();
q.pop();
```

或者：

```
int x=q.top();  
q.pop();
```

---

## lower\_bound 返回 end

```
int pos=lower_bound(a.begin(),a.end(),x)-a.begin();
```

如果 `x` 比所有元素都大，`pos==a.size()`。

访问前必须判断：

```
if(pos<a.size())
```

---

## cmp 不要写 `<=` 或 `>=`

错误：

```
return a<=b;
```

正确：

```
return a<b;
```

或者：

```
return a>b;
```

---

## 前缀和区间公式

```
s[r]-s[l-1]
```

不要写成：

```
s[r]-s[l]
```

---

## 差分右边界

---

```
d[r+1]-=c;
```

不是：

```
d[r]-=c;
```

---

## 字符转数字

---

```
int x=c-'0';
```

不要直接写：

```
int x=c;
```

---

## 位运算判断奇偶

---

```
if(x&1)
```

可以判断奇数。

如果写复杂判断，建议加括号：

```
if((x&1)==1)
```

---

## C++ 里 `^` 不是乘方

---

```
2^3
```

不是 8。

这是异或。

如果要表示  $2^3$ ：

